# CSE 112 Computer Organization 2025: Mid-Semester Exam

**Total Time: 60 Minutes** **Maximum: 30 Points**

**Instructions:**

1. **Write all the assumptions clearly, if any.** ***Only reasonable assumptions will be considered if any ambiguity is found in the question.***
2. **No electronic gadgets apart from calculators are allowed.**

**ISA description:**

| First Block Instructions | Description | Operation |
|---|---|---|
| addi rd,rs, imm[11:0] | Add immediate | rd = rs + sext(imm[11:0]) |
| sub rd, rs1, rs2 | Subtract registers | rd = rs1 - rs2 |
| add rd, rs1, rs2 | Add registers | rd = rs1 + rs2 |
| xor rd, rs1, rs2 | Perform XOR boolean operation | rd = rs1 xor rs2 |
| slt rd, rs1, rs2 | Set Less than | rd = 1. If signed(rs1) < signed(rs2) else rd=0 |
| sltu rd,rs1,rs2 | Set Less than unsigned | rd = 1. If unsigned(rs1) < unsigned(rs2) else rd=0 |
| lw rd,#imm[11:0](rs1) | Load Word | rd=mem(sext(rs1+imm)) |
| sw rd,#imm[11:0](rs1) | Store Word | mem(sext(rs1+imm))=rd |
| beq rs1, rs2, imm[12:1] | Branch if equal | PC = PC + sext({imm[12:1],1'b0}) if rs1 == rs2 |
| **beq x0,x0,#0** | Branch if equal | **HALT** |

| Second Block Instructions | Description | Operation |
|---|---|---|
| bne rs1, rs2, imm[12:1] | Branch if not equal | PC = PC + sext({imm[12:1],1'b0}) if rs1 =! rs2 |
| bltu rs1,rs2,imm[12:1] | Branch if less than unsigned | PC = PC + sext({imm[12:1],1'b0}) If unsigned(rs1) < unsigned(rs2) |
| blt rs1,rs2,imm[12:1] | Branch if Less than | PC = PC + sext[imm[12:1],1'b0}] If signed(rs1) < signed(rs2) |
| bge rs1,rs2, imm[12:1] | Branch if Greater than or Equal | PC= PC + sext({imm[12:1],1'b0}) If signed(rs1) > signed(rs2) |
| bgeu rs1,rs2, imm[12:1] | Branch if Greater than or equal unsigned | PC= PC + sext({imm[12:1],1'b0}) If unsigned(rs1) > unsigned(rs2) |

***Note:*** The ISA is a subset of RISC-V ISA. Each instruction is of 32-bit size, we have 32 registers as standard from x0 to x31 with x0 hardwired to zero. The **symbol #** indicates an immediate value in the decimal format.

**Q1. [CO2] Assembly Execution:**

We need to answer how many times the character * will be printed and what is the value in registers {x1,x2} after execution of every instruction. **[5 points]**

| Assembly Program | |
|---|---|
| xor x2,x2,x2 | **Instruction:** OUT * |
| addi x2,x2,#10 | Explanation: This will print * at the output. And, get to the next line. |
| **LABEL:** out * | |
| sltu x1,x0,x2 | |
| sub x2,x2,x1 | |
| bne x2,x0,**LABEL** | |
| **HERE:** beq x0,x0,**HERE** | |

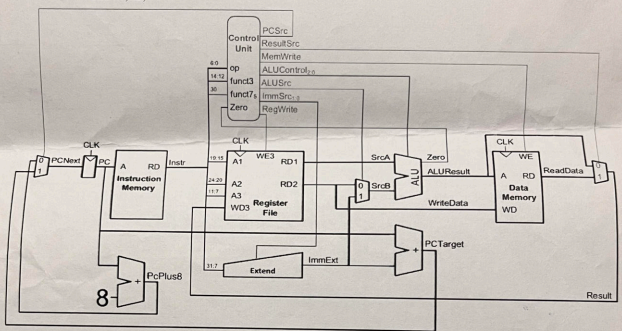
**Q2. [CO2] Assembly Instruction Encoding:** **[10 points]**

| | Pseudo Code |
|---|---|
| We need to convert the pseudo code given on the right side into an assembly program using the above-mentioned ISA. And, we do need to report the final value of the register **E** (corresponding mapped register) after execution of every instruction. | A=20, B=11, D=1, E=1<br>while(A!=B)<br>{<br>A=A-D<br>E=E+1<br>}<br>HALT |

**Q3. [CO3, CO2] Student_A** designed a processor microarchitecture for the instructions mentioned in the **First block**. **Student_A** intentionally changed the default PC update hardware (as depicted in the below block diagram). The designed processor uses a 32-bit data width bus and a byte addressable instruction memory. **Student_B** is **unaware** about the changes made by **Student_A** in the PC update hardware. **Student_B** wrote the below-mentioned assembly code to execute on the processor designed by **Student_A**.

**Assembly Code:**

```
addi x2,x0,#11
add  x3, x2, x0
sub x4, x5, x2
add x1, x3, x4
addi x1, x3, #4
```




Answer the following questions:

a) List out the instructions executed by the microarchitecture and the value of the registers(x0 to x5) after each instruction.

b) Does the microarchitecture designed by **Student_A** align with the intent of the code written by **Student_B**. Give reason.

c) How will **Student_A** rectify the microarchitecture to solve the issue?

Assume that the **initial** values of registers are as x1→1;...; x31→31.

**[3+2+1=6 points]**

**Q4. [CO3, CO2]** If the microarchitecture described in the above question has the following control signal.

1. **RegWrite**: If the instruction writes into the register, the value is 1, else it is 0.
2. **ALUSrc:** When the value is 0, the value of register 2 is read, else the extended immediate value is read.
3. **MemWrite:** If the instruction writes into the data memory, the value is 1, else it is 0.
4. **ResultSrc**: When the value is 0, output from the execute stage is written back. When 1, output from the memory stage is written back.
5. **ALUControl**: 000 for R-type ADD instruction, 001 for R-type SUB instruction, 010 for instructions using immediate values in any form, 011 for instructions implementing boolean operations.

Assuming the instructions listed below are performed, determine the value of the control signals. Use "**x**" as a don't care value wherever necessary. Answer in the form of the given table:

**[9 points]**

| | RegWrite | ALUSrc | MemWrite | ALUControl | ResultSrc |
|---|---|---|---|---|---|
| add x3,x2,x1 | | | | | |
| lw x2, #4(x3) | | | | | |
| sw x1, #8(x5) | | | | | |
| addi x2, x1, #8 | | | | | |
| sub x1,x2,x3 | | | | | |
| beq x1, x2, #0 | | | | | |